

Nirob Library: Project Proposal

Oct 1, 2026 · @arafat



Northern University Bangladesh

Department of Computer Science and Engineering

Project Proposal: Nirob Library, a Web-Based Library Management System

Submitted by	Ahosanul Haque Nirob
Student ID	41240202244
Department	Computer Science and Engineering
University	Northern University Bangladesh
Course	<i>(course code and title)</i>
Submitted to	<i>(supervisor name and designation)</i>

Table of contents

1. Abstract
2. Introduction and problem statement

3. Objectives
4. Literature review
5. Feasibility study
6. Scope and features
7. Requirements
8. Use case diagram
9. Methodology and system architecture
10. Borrow and return process
11. Database design
12. Tools and technologies
13. Security measures
14. Testing plan
15. Work plan
16. Expected outcomes, limitations and future work
17. Conclusion
18. References

Abstract

Nirob Library is a web application that replaces paper registers with one online system for books, authors, categories, members and loans. Members sign up, browse the catalogue and borrow or return books themselves. An administrator manages every record and issues books to members.

The system is built with PHP, MySQL and Bootstrap 5, with a Laravel project for the database structure and demo data. It tracks available copies automatically, gives a 14-day due date on every loan, marks late loans as overdue and charges a late fine of 5 Taka per day.

Introduction and problem statement

Many small and institutional libraries still record loans in paper registers or spreadsheets. This makes daily work slow and error-prone:

- **No live stock count.** Staff cannot see at a glance how many copies of a book are on the shelf.
- **Missed due dates.** Overdue loans are found only when someone checks the register by hand, so fines are applied unevenly.

- **Slow searching.** Finding a book by title, author, category or ISBN means going through pages or files.
- **No self-service.** Members must visit or call the library to learn what is available or what they have borrowed.
- **Scattered records.** Book, author, member and loan details live in separate places and drift out of sync.

Nirob Library addresses these problems with one shared database and a browser interface for both members and the administrator.

Objectives

1. Build a secure login and registration system with separate member and administrator access.
2. Let members search the catalogue and borrow or return books online.
3. Keep the count of available copies correct automatically on every loan and return.
4. Set due dates, detect overdue loans and calculate late fines without manual work.
5. Give the administrator full add, edit and delete control over books, authors, categories and members.
6. Show live statistics on a dashboard: total books, available titles, active loans and overdue loans.
7. Design a normalised relational database and document it with an ER diagram.

Literature review

Libraries today use paper registers, spreadsheets or a full integrated library system such as [Koha](#). Registers and spreadsheets are cheap but slow and error-prone; full systems are powerful but need a dedicated server and trained staff. The table compares them with Nirob Library.

Approach	Search	Live stock count	Member self-service	Overdue and fines	Setup effort
Paper register	Manual	No	No	Manual	None
Spreadsheet	Basic	Manual	No	Manual formulas	Low
Full library system (Koha)	Advanced	Yes	Yes	Yes	High
Nirob Library	By title, author, category, ISBN	Yes, automatic	Yes	Yes, automatic	Low (XAMPP)

Nirob Library keeps the automatic features of a full system but stays small enough to install on one computer with XAMPP.

Feasibility study

The project is feasible on all four counts. **Technically**, PHP, MySQL, Apache and Bootstrap are mature, taught in the CSE curriculum and installed in one step with XAMPP on any computer with 4 GB of RAM. **Economically**, every tool is free and open source, so there is no licence cost. **Operationally**, the interface uses familiar web forms, so members and staff need little training. **On schedule**, the work fits six two-week phases, each ending with a working, tested part.

Scope and features

The system has two roles: **members**, who borrow books, and the **administrator**, who manages the library's records.

Members register, search the catalogue, borrow books for 14 days and return them. The administrator can also add, edit and delete books, authors, categories and members, issue books to any member and view every borrow record. Each feature is listed as a numbered requirement in the next section.

Out of scope: online payment of fines, email reminders, barcode scanning and book cover image uploads.

Requirements

Functional requirements

ID	Requirement	User
FR-01	Register with name, email, phone and password; email must be unique	Member
FR-02	Sign in with email and password, and sign out	All
FR-03	Show a dashboard with book, author, category and loan counts	All
FR-04	Search books by title, author, category or ISBN	All
FR-05	Borrow an available book for 14 days	Member

ID	Requirement	User
FR-06	Refuse a loan when no copy is left, the membership is not active, or the member already holds the same book	System
FR-07	List the member's own loans with due date, days left and status	Member
FR-08	Return a book and add a late fine of 5 Taka per overdue day	Member, Admin
FR-09	Mark every loan past its due date as overdue	System
FR-10	Add, edit and delete books, authors and categories	Admin
FR-11	Add, edit and delete members, including membership dates and status	Admin
FR-12	Issue a book to any active member	Admin
FR-13	View all borrow records and filter them by active, overdue or returned	Admin
FR-14	Block deleting a book on loan, a member with unreturned books, or an author or category that still has books	System

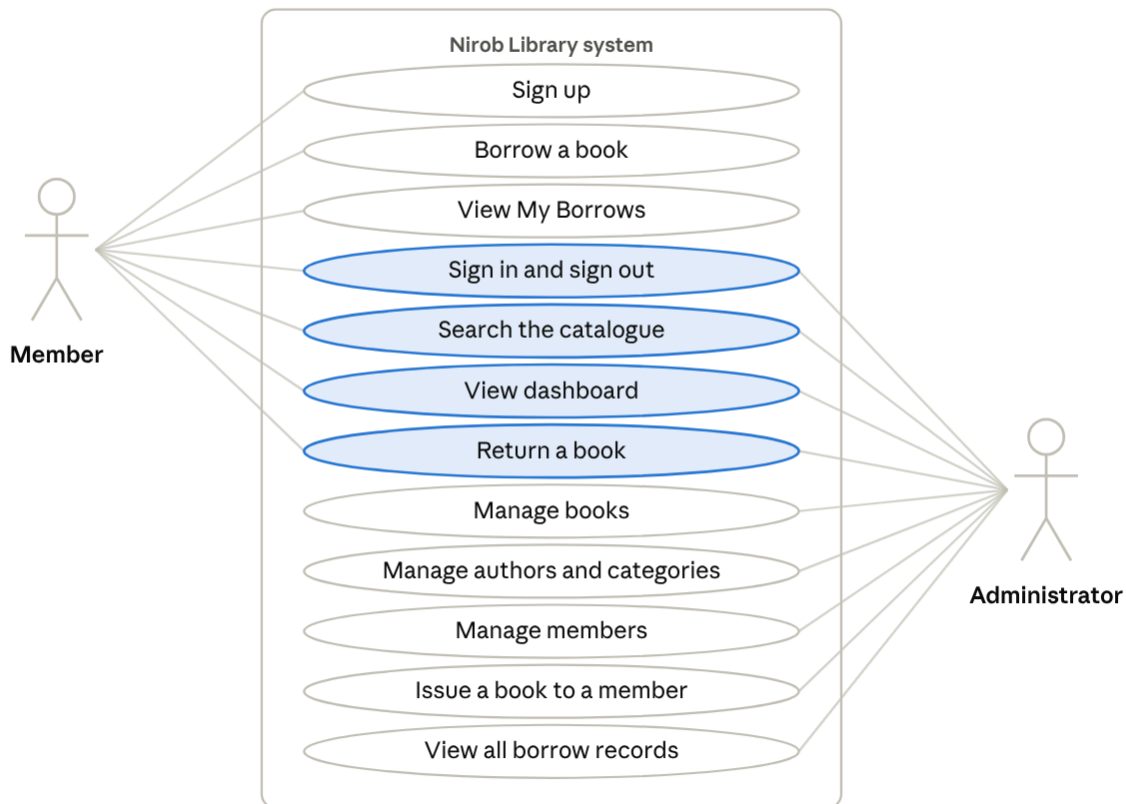
Non-functional requirements

- **Security:** passwords stored only as bcrypt hashes; all database queries use prepared statements; admin actions are checked on the server.
- **Usability:** every page works on a phone, tablet and desktop; forms show clear error messages.
- **Performance:** pages and searches respond in under 2 seconds on a local server with the demo data.
- **Reliability:** each loan and return runs inside a database transaction, so copy counts never go wrong, even when two people click at once.
- **Maintainability:** one API file, clear table design and Laravel migrations make the system easy to extend.

Use case diagram

Two actors use the system. Members serve themselves; the administrator can do everything a member can, plus manage the library's records.

Members borrow and return books; the administrator also manages every record



Shaded use cases are shared by both actors.

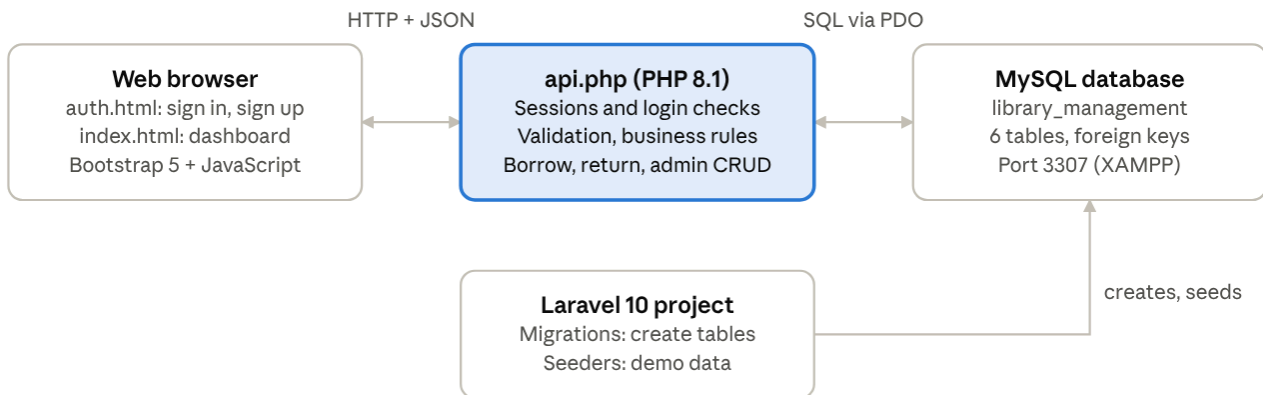
use case diagram · 2 actors, 12 use cases

The four shaded use cases in the middle are available to both actors; the top three are member-only and the bottom five are admin-only.

Methodology and system architecture

The system follows a three-tier design: browser pages, a PHP API, and a MySQL database. The pages never touch the database directly; every read and write goes through `api.php`, which checks the session, validates input and runs each loan or return inside a database transaction.

Pages call one PHP API, which alone talks to MySQL



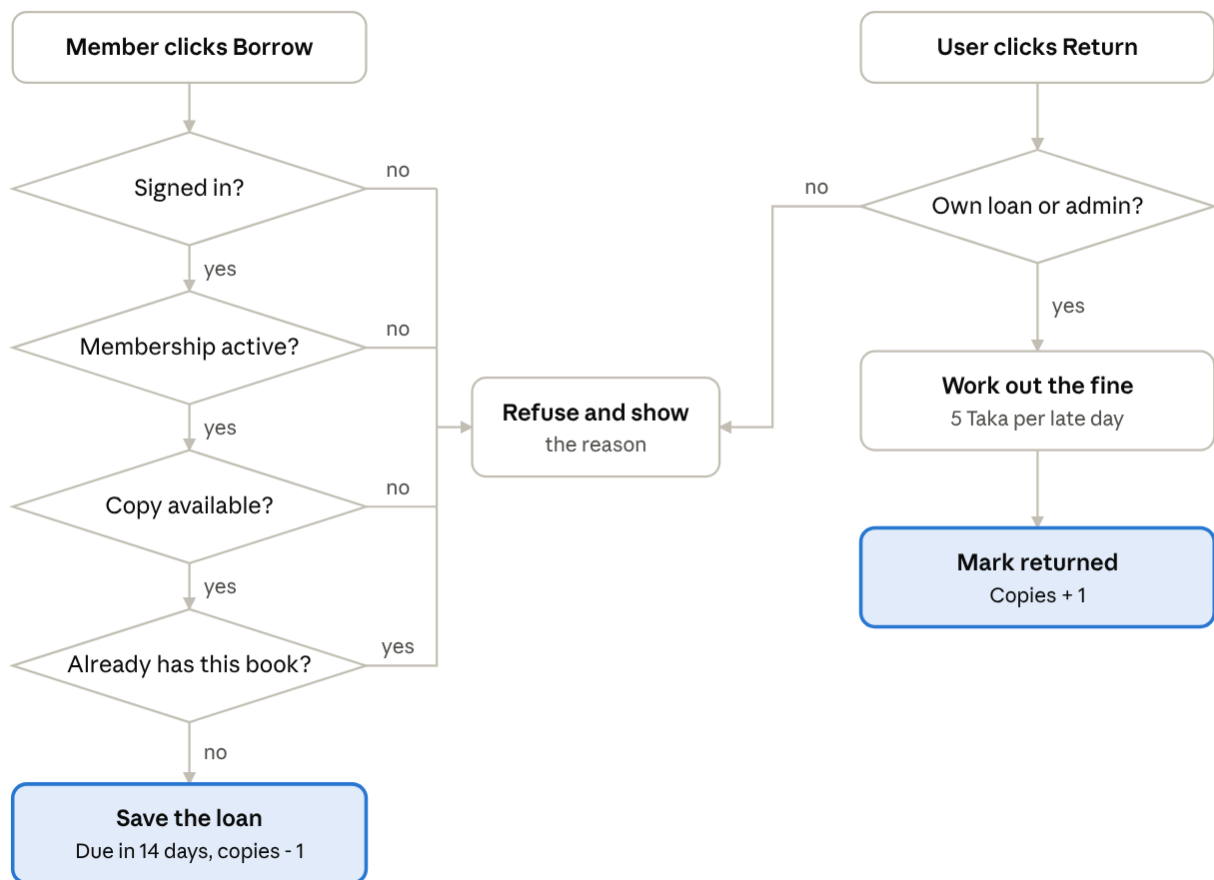
system architecture · 3 tiers + Laravel setup

The Laravel project defines the tables through migrations and fills them with demo data through seeders. Development follows an incremental model: database first, then the API, then the member pages, then the admin tools, testing each step against the real database.

Borrow and return process

A loan is saved only after four checks pass; a return is accepted only from the member who borrowed the book or from the administrator.

Every borrow passes four checks before the loan is saved



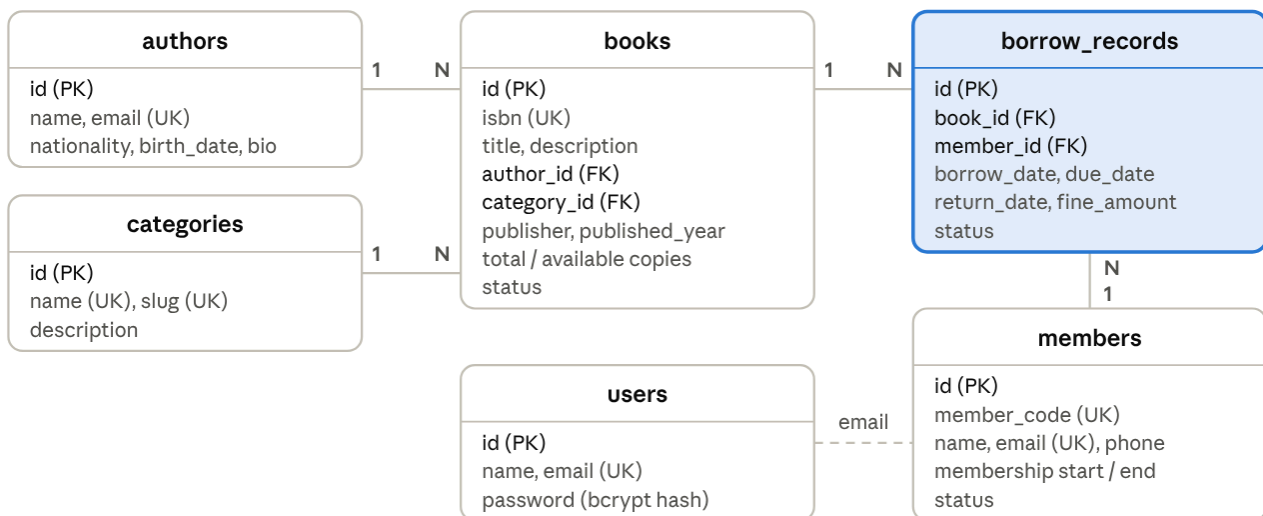
borrow and return flow · 5 checks

Both flows run inside a database transaction that locks the book's row, so two members can never take the last copy at the same moment.

Database design

The MySQL database `library_management` holds six tables linked by four foreign keys. Books and members have a many-to-many relationship, resolved by the `borrow_records` table: one row per loan.

Borrow records link members to books; each book has one author and one category



PK primary key · FK foreign key · UK unique · 1 / N = one-to-many · dashed = matched by email, no foreign key

ER diagram · 6 tables, 4 foreign keys

Books, authors and members use soft deletes (a `deleted_at` date), so removed records stay in the database and loan history is kept. A user account is matched to a member profile by email; one is created automatically for a user who has none.

Tools and technologies

Layer	Technology	Purpose
Front end	HTML5, CSS3, JavaScript (Fetch API)	Pages, forms and live updates without reloading
UI framework	Bootstrap 5.3, Bootstrap Icons, Space Grotesk font	Responsive layout, tables, pop-up forms, dark theme
Back end	PHP 8.1 with PDO	JSON API, sessions, validation and business rules
Framework	Laravel 10	Migrations, Eloquent models and database seeders
Database	MySQL 8 (XAMPP, port 3307)	Stores all library data with foreign keys
Web server	Apache (XAMPP)	Serves the pages and the API locally

Layer	Technology	Purpose
Security	bcrypt password hashing, prepared statements, HTML escaping	Protects passwords and blocks SQL injection and XSS
Tools	VS Code, Composer, phpMyAdmin, Git	Coding, packages, database inspection, version control

Security measures

Threat	Protection in Nirob Library
Stolen passwords	Passwords are stored only as bcrypt hashes, never as plain text
SQL injection	Every query uses PDO prepared statements
Cross-site scripting (XSS)	All data shown on a page is HTML-escaped
Session hijacking	The session ID is regenerated on sign-in and destroyed on sign-out
Members using admin functions	Every admin action is checked on the server
Wrong copy counts	Loans and returns run in transactions that lock the book row

Testing plan

Testing is done at three levels: **unit testing** of each API action with sample requests, **integration testing** of the pages against the real MySQL database, and **user acceptance testing** with one member and one administrator account. The main test cases are:

ID	Test case	Input	Expected result
TC-01	Valid sign-in	Correct email and password	Dashboard opens with the user's name
TC-02	Invalid sign-in	Wrong password	"Invalid email or password" message
TC-03	Borrow an available book	Book with 3 of 5 copies free	Loan saved, due in 14 days, 2 copies left

ID	Test case	Input	Expected result
TC-04	Borrow with no copies left	Book with 0 copies free	Borrow button disabled; server refuses
TC-05	Return on time	Loan returned before due date	Status returned, fine 0, copies back up by 1
TC-06	Return late	Loan returned 3 days after due date	Fine of 15 Taka recorded
TC-07	Duplicate ISBN	ISBN of an existing book	"Same ISBN already exists" message
TC-08	Member opens admin action	Member session calls the save action	Server answers "Admins only"

Work plan

The project is planned over 12 weeks in six two-week phases, each ending with a working, tested part of the system.

Six two-week phases take the project from design to final demo in 12 weeks

Requirements and design

Use cases, ER diagram, UI plan

Wk 1-2

Database and demo data

Migrations, seeders, MySQL setup

Wk 3-4

Back-end API

Login, sessions, borrow and return

Wk 5-6

Member features

Sign up, catalogue, My Borrows

Wk 7-8

Admin features

CRUD forms, issue books, records

Wk 9-10

Testing and report

Test cases, fixes, final report

Wk 11-12

Final demo and submission ◆

proposed work plan · 6 phases, 12 weeks

The week numbers are a proposed schedule and can be adjusted to the semester calendar.

Expected outcomes, limitations and future work

Outcomes: a working system loaded with demo data (32 books, 20 authors, 10 categories, 20 members, 21 loans), automatic copy counts, due dates and fines, and a documented database design.

Limitations: it runs on a local XAMPP server, the administrator is fixed by email address, and fines are paid offline.

Future work: email or SMS due-date reminders, book cover uploads with barcode scanning, and online fine payment through bKash or Nagad.

Conclusion

Nirob Library turns a paper-based lending process into one online system that members and staff can use from a browser. It keeps stock counts, due dates and fines accurate automatically and gives the administrator full control of the catalogue and membership. The project applies core Computer Science and Engineering topics: relational database design, web back-end development, authentication and secure coding, and responsive user interface design.

References

1. Laravel. *Laravel 10.x Documentation*. laravel.com/docs/10.x
2. The PHP Group. *PHP Manual: PDO and password hashing*. php.net/manual/en
3. Oracle. *MySQL 8.0 Reference Manual*. dev.mysql.com/doc/refman/8.0/en
4. Bootstrap. *Bootstrap 5.3 Documentation*. getbootstrap.com/docs/5.3
5. Apache Friends. *XAMPP*. apachefriends.org
6. Koha Community. *Koha Library Software*. koha-community.org
7. OWASP Foundation. *OWASP Top Ten*. owasp.org/www-project-top-ten